



- Name : Harshvardhan Patil
- Branch : AI&DS
- Roll No : SY B 23
- Batch : SY B2

AIM :

Perform Data Ingestion and Cleaning: Acquire data and prepare it for analysis.

THEORY

1. What is data ingestion and why is it important in data science? Which Python library is commonly used for data ingestion and cleaning?

Data ingestion is collecting data from various sources into a system for processing and analysis. It's vital in data science for providing high-quality, structured data, forming the basis for all subsequent analysis and model building.

pandas is commonly used for data ingestion and cleaning in Python.

2. Why is data cleaning necessary before performing analysis or machine learning? Data cleaning is essential for several reasons:

Data cleaning is essential because raw data often contains errors, inconsistencies, or outliers. Clean data ensures accurate analysis, reliable insights, better model performance, and consistency across diverse data sources, ultimately leading to more efficient and trustworthy results.

3. How do you load a CSV file into a Pandas DataFrame? You can load a CSV file into a Pandas DataFrame using the `pd.read_csv()` function:

```
import pandas as pd
```

```
df = pd.read_csv('your_file_name.csv')
```

4. What is the difference between `head()` and `tail()` functions in Pandas? Both `head()` and `tail()` are DataFrame methods used to quickly inspect the rows of a DataFrame:

`df.head(n=5)`: Returns the first `n` rows of the DataFrame. By default, it returns the first 5 rows.

`df.tail(n=5)`: Returns the last `n` rows of the DataFrame. By default, it returns the last 5 rows.

5. How can you identify missing values in a dataset? You can identify missing values in a Pandas DataFrame using the `isnull()` (or `isna()`) method, often combined with `sum()` to get a count:

```
import pandas as pd
```

Use `df.isnull().sum()` to count missing values per column

`df.isnull().sum().sum()` for the total missing values in the DataFrame.

`df.isnull()` returns a boolean DataFrame indicating where values are missing.

```
import pandas as pd
data = {'col1': [1, 2, None, 4], 'col2': ['A', 'B', 'C', None]}
```

```
df = pd.DataFrame(data)
print(df.isnull().sum())
```

```
In [ ]: import pandas as pd

# 1. Load CSV file and display first 5 rows
df = pd.read_csv("/content/employee_performance_data (2).csv")
print("First 5 rows:")
display(df.head(5))

# 2. Check missing values
print("\nMissing values:")
display(df.isnull().sum())

# 3. Fill missing 'Bonus' with 0
df['Bonus'] = df['Bonus'].fillna(0)

# 4. Convert column names to lowercase
df.columns = df.columns.str.lower()
print("\nColumn names:")
display(df.columns.tolist())

# 5. Display dataset shape
print("\nShape:", df.shape)

# 6. Remove duplicate rows
df = df.drop_duplicates()
print("\nDuplicates removed")

# 7. Handle missing values
df['salary'] = df['salary'].fillna(df['salary'].mean())
```

```

df['experience'] = df['experience'].fillna(df['experience'].mean())
df['performance_rating'] = df['performance_rating'].fillna(df['performance_rating'].mean())
df['work_hours_per_week'] = df['work_hours_per_week'].fillna(df['work_hours_per_week'].mean())
df['bonus'] = df['bonus'].fillna(df['bonus'].mean())

print("\nMissing values after cleaning:")
display(df.isnull().sum())

# 8. Convert join_date to datetime
df['join_date'] = pd.to_datetime(df['join_date'])

# 9. Filter employees joined after 2015
filtered_df = df[df['join_date'].dt.year > 2015]
print("\nFiltered data:")
display(filtered_df)

# 10. Save cleaned dataset
df.to_csv("employee_cleaned_data.csv", index=False)
print("\nData saved successfully")

```

First 5 rows:

	Employee_ID	Name	Age	Gender	Department	Join_Date	Salary	Experier
0	101	Alice	50	Female	Marketing	2009-01-18	62606.0	1
1	102	Bob	36	Female	Marketing	2012-01-01	41534.0	
2	103	Charlie	29	Female	IT	2007-02-15	NaN	
3	104	David	42	Male	Marketing	2015-12-29	31016.0	
4	105	Eve	40	Female	IT	2005-02-04	119789.0	

Missing values:

	0
Employee_ID	0
Name	0
Age	0
Gender	0
Department	0
Join_Date	0
Salary	1
Experience	1
Performance_Rating	1
Bonus	1
Work_Hours_Per_Week	1

dtype: int64

Column names:

```
[ 'employee_id',
  'name',
  'age',
  'gender',
  'department',
  'join_date',
  'salary',
  'experience',
  'performance_rating',
  'bonus',
  'work_hours_per_week']
```

Shape: (20, 11)

Duplicates removed

Missing values after cleaning:

	0
employee_id	0
name	0
age	0
gender	0
department	0
join_date	0
salary	0
experience	0
performance_rating	0
bonus	0
work_hours_per_week	0

dtype: int64

Filtered data:

	employee_id	name	age	gender	department	join_date	salary	experien
13	114	Nina	23	Female	HR	2019-04-23	42185.0	8.
16	117	Quinn	59	Female	HR	2018-02-20	69099.0	1.
18	119	Steve	42	Male	HR	2016-10-07	68044.0	5.

Data saved successfully

CONCLUSION

This practical successfully demonstrated core data ingestion and cleaning techniques using the `pandas` library. We covered loading data, identifying and handling missing values (using imputation and zero-filling), standardizing column names, removing duplicates, converting data types, and filtering data based on conditions. The cleaned dataset was then saved, illustrating the essential steps required to transform raw data into a reliable format ready for analysis or machine learning applications.